

RAJARATA UNIVERSITY OF SRI LANKA

FACULTY OF TECHNOLOGY

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

ICT 3219 – Mobile Application Development

Practical Sheet – 05

Introduction to SQLite Databases in Android

Objectives

- Understand what SQLite is and why local storage is needed in mobile apps
- Create and manage a database using SQLiteOpenHelper
- Perform CRUD operations (Create, Read, Update, Delete) from Java
- Display database records dynamically in a ListView
- Build, run and test a simple database-backed app on an emulator or device

Why SQLite?

Every Android device ships with SQLite, a lightweight, file-based relational database built into the operating system. Unlike variables in memory, data saved to SQLite survives when the app is closed, the phone is restarted, or the activity is destroyed. It is the standard choice for storing structured data locally, such as a list of records, settings, or offline content.

Task 1: Project Setup

1. Open Android Studio
2. Select File → New → New Project
3. Choose Empty Views Activity template
4. Name: Student Records
5. Package name: com.example.studentrecords
6. Language: Java
7. Minimum SDK: API 24 (Android 7.0)

Task 2: Understand the Database Layer

In Android, you don't talk to SQLite directly — you extend a helper class called SQLiteOpenHelper. It manages creating the database file, creating tables the first time the app runs, and upgrading the schema later if the table structure changes.

- onCreate(db) — runs once, the first time the database is created; this is where you write CREATE TABLE statements
- onUpgrade(db, oldVersion, newVersion) — runs when DATABASE_VERSION is increased; typically drops and recreates tables
- getWritableDatabase() / getReadableDatabase() — open the database ready for writes or reads

Task 3: Create DatabaseHelper.java

Right-click the com.example.studentrecords package → New → Java Class → name it DatabaseHelper. This class defines the students table and provides methods to add, read, and delete records.

```
package com.example.studentrecord;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

public class DatabaseHelper extends SQLiteOpenHelper {

    private static final String DATABASE_NAME = "StudentDB";
    private static final int DATABASE_VERSION = 2; // bumped: new columns added

    public static final String TABLE_NAME = "students";
    public static final String COLUMN_ID = "_id";
    public static final String COLUMN_NAME = "name";
    public static final String COLUMN_AGE = "age";
    public static final String COLUMN_INDEX_NUMBER = "index_number";
    public static final String COLUMN_REG_NUMBER = "registration_number";
    public static final String COLUMN_SUBJECT_CODE = "subject_code";
    public static final String COLUMN_RESULT = "result";

    public DatabaseHelper(Context context) {
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        String createTable = "CREATE TABLE " + TABLE_NAME + " (" +
                COLUMN_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +
                COLUMN_NAME + " TEXT NOT NULL, " +
                COLUMN_AGE + " INTEGER NOT NULL, " +
                COLUMN_INDEX_NUMBER + " TEXT NOT NULL, " +
                COLUMN_REG_NUMBER + " TEXT NOT NULL, " +
                COLUMN_SUBJECT_CODE + " TEXT NOT NULL, " +
                COLUMN_RESULT + " TEXT)";
        db.execSQL(createTable);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME);
        onCreate(db);
    }

    public boolean addStudent(String name, int age, String indexNumber,
                              String regNumber, String subjectCode, String result) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues values = new ContentValues();
        values.put(COLUMN_NAME, name);
        values.put(COLUMN_AGE, age);
```

```
        values.put(COLUMN_INDEX_NUMBER, indexNumber);
        values.put(COLUMN_REG_NUMBER, regNumber);
        values.put(COLUMN_SUBJECT_CODE, subjectCode);
        values.put(COLUMN_RESULT, result);
        long result_ = db.insert(TABLE_NAME, null, values);
        db.close();
        return result_ != -1;
    }

    public Cursor getAllStudents() {
        SQLiteDatabase db = this.getReadableDatabase();
        return db.query(TABLE_NAME, null, null, null, null, null, COLUMN_ID + " DESC");
    }

    public boolean deleteStudent(long id) {
        SQLiteDatabase db = this.getWritableDatabase();
        String[] args = new String[]{String.valueOf(id)};
        int rows = db.delete(TABLE_NAME, COLUMN_ID + "=?", args);
        db.close();
        return rows > 0;
    }

    public void deleteAllStudents() {
        SQLiteDatabase db = this.getWritableDatabase();
        db.execSQL("DELETE FROM " + TABLE_NAME);
        db.close();
    }
}
```

Column reference:

Column constant	Column name	Type	Notes
COLUMN_ID	_id	INTEGER PK	Auto-increment primary key
COLUMN_NAME	name	TEXT	Student name
COLUMN_AGE	age	INTEGER	Student age
COLUMN_INDEX_NUMBER	index_number	TEXT	New — university index number
COLUMN_REG_NUMBER	registration_number	TEXT	New — registration number
COLUMN_SUBJECT_CODE	subject_code	TEXT	New — e.g. ICT3219
COLUMN_RESULT	result	TEXT	New — grade / result for the subject

Task 4: Design the UI (activity_main.xml)

The layout now includes four extra input fields, and the whole screen uses a lighter, more pleasant colour palette instead of the original deep purple. First define the palette in res/values/colors.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="colorBackground">#FDFBFF</color>   <!-- near-white lavender -->
    <color name="colorPrimary">#B8A9E8</color>       <!-- soft lavender -->
    <color name="colorPrimaryDark">#9C8FD1</color>
    <color name="colorAccentMint">#A8E6CF</color>    <!-- soft mint accent -->
    <color name="colorButton">#C3B7EA</color>        <!-- light lavender button -->
    <color name="colorButtonDelete">#F3B8B8</color>  <!-- soft coral for delete -->
    <color name="colorText">#4A4458</color>          <!-- readable dark plum -->
    <color name="colorCardBackground">#F4F1FB</color>
</resources>
```

Build the layout below in res/layout/activity_main.xml. It uses a vertical LinearLayout with two EditText fields for input, an Add button, a ListView to display saved records, and a Delete All button.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/colorBackground"
    android:padding="24dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Student Records"
        android:textColor="@color/colorText"
        android:textSize="22sp"
        android:textStyle="bold"
        android:layout_marginBottom="16dp"/>

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Student Name"
        android:inputType="textPersonName"/>

    <EditText
        android:id="@+id/etAge"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Age"
        android:inputType="number"
        android:layout_marginTop="12dp"/>

    <EditText
        android:id="@+id/etIndexNumber"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Index Number"
        android:inputType="text"
        android:layout_marginTop="12dp"/>
```

```
    <EditText
        android:id="@+id/etRegNumber"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Registration Number"
        android:inputType="text"
        android:layout_marginTop="12dp"/>

    <EditText
        android:id="@+id/etSubjectCode"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Subject Code (e.g. ICT3219)"
        android:inputType="textCapCharacters"
        android:layout_marginTop="12dp"/>

    <EditText
        android:id="@+id/etResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Result"
        android:inputType="text"
        android:layout_marginTop="12dp"/>

    <Button
        android:id="@+id/btnAdd"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Add Student"
        android:backgroundTint="@color/colorButton"
        android:textColor="@color/colorText"
        android:layout_marginTop="20dp"/>

    <ListView
        android:id="@+id/lvStudents"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:background="@color/colorCardBackground"
        android:divider="@color/colorPrimary"
        android:dividerHeight="1dp"
        android:padding="8dp"
        android:layout_marginTop="20dp"/>

    <Button
        android:id="@+id/btnDeleteAll"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Delete All"
        android:backgroundTint="@color/colorButtonDelete"
        android:textColor="@color/colorText"
        android:layout_marginTop="12dp"/>

</LinearLayout>
```

Note: the ListView needs layout_height="0dp" with layout_weight="1" so it fills the remaining space instead of being pushed off-screen by the keyboard.

Task 5: Implement MainActivity.java

This class connects to the widgets, reads and validates input, writes to the database through DatabaseHelper, and refreshes the ListView using an ArrayAdapter every time the data changes.

```java
package com.example.studentrecord;

import androidx.appcompat.app.AppCompatActivity;

import android.database.Cursor;
import android.os.Bundle;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ListView;
import android.widget.Toast;

import com.example.studentrecord.DatabaseHelper;

import java.util.ArrayList;
import java.util.List;

public class MainActivity extends AppCompatActivity {

    private EditText etName, etAge, etIndexNumber, etRegNumber, etSubjectCode,
etResult;
    private Button btnAdd, btnDeleteAll;
    private ListView lvStudents;
    private DatabaseHelper dbHelper;
    private ArrayAdapter<String> adapter;
    private final List<String> studentList = new ArrayList<>();
    private final List<Long> studentIds = new ArrayList<>();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Link variables to XML widgets
        etName = findViewById(R.id.etName);
        etAge = findViewById(R.id.etAge);
        etIndexNumber = findViewById(R.id.etIndexNumber);
        etRegNumber = findViewById(R.id.etRegNumber);
        etSubjectCode = findViewById(R.id.etSubjectCode);
        etResult = findViewById(R.id.etResult);
        btnAdd = findViewById(R.id.btnAdd);
        btnDeleteAll = findViewById(R.id.btnDeleteAll);
        lvStudents = findViewById(R.id.lvStudents);

        dbHelper = new DatabaseHelper(this);
        adapter = new ArrayAdapter<>(this, android.R.layout.simple_list_item_1,
studentList);
        lvStudents.setAdapter(adapter);

        loadStudents();

        btnAdd.setOnClickListener(v -> addStudent());

        btnDeleteAll.setOnClickListener(v -> {
            dbHelper.deleteAllStudents();
            loadStudents();
            Toast.makeText(this, "All records deleted", Toast.LENGTH_SHORT).show();
        });

        // Long-press a row to delete just that record
        lvStudents.setOnItemLongClickListener((parent, view, position, id) -> {
            long studentId = studentIds.get(position);
            dbHelper.deleteStudent(studentId);
            loadStudents();
            Toast.makeText(this, "Record deleted", Toast.LENGTH_SHORT).show();
```

```
            return true;
        });
    }

    private void addStudent() {
        String name = etName.getText().toString().trim();
        String ageStr = etAge.getText().toString().trim();
        String indexNumber = etIndexNumber.getText().toString().trim();
        String regNumber = etRegNumber.getText().toString().trim();
        String subjectCode = etSubjectCode.getText().toString().trim();
        String result = etResult.getText().toString().trim();

        if (name.isEmpty() || ageStr.isEmpty() || indexNumber.isEmpty()
                || regNumber.isEmpty() || subjectCode.isEmpty()) {
            Toast.makeText(this, "Please fill in all required fields",
Toast.LENGTH_SHORT).show();
            return;
        }

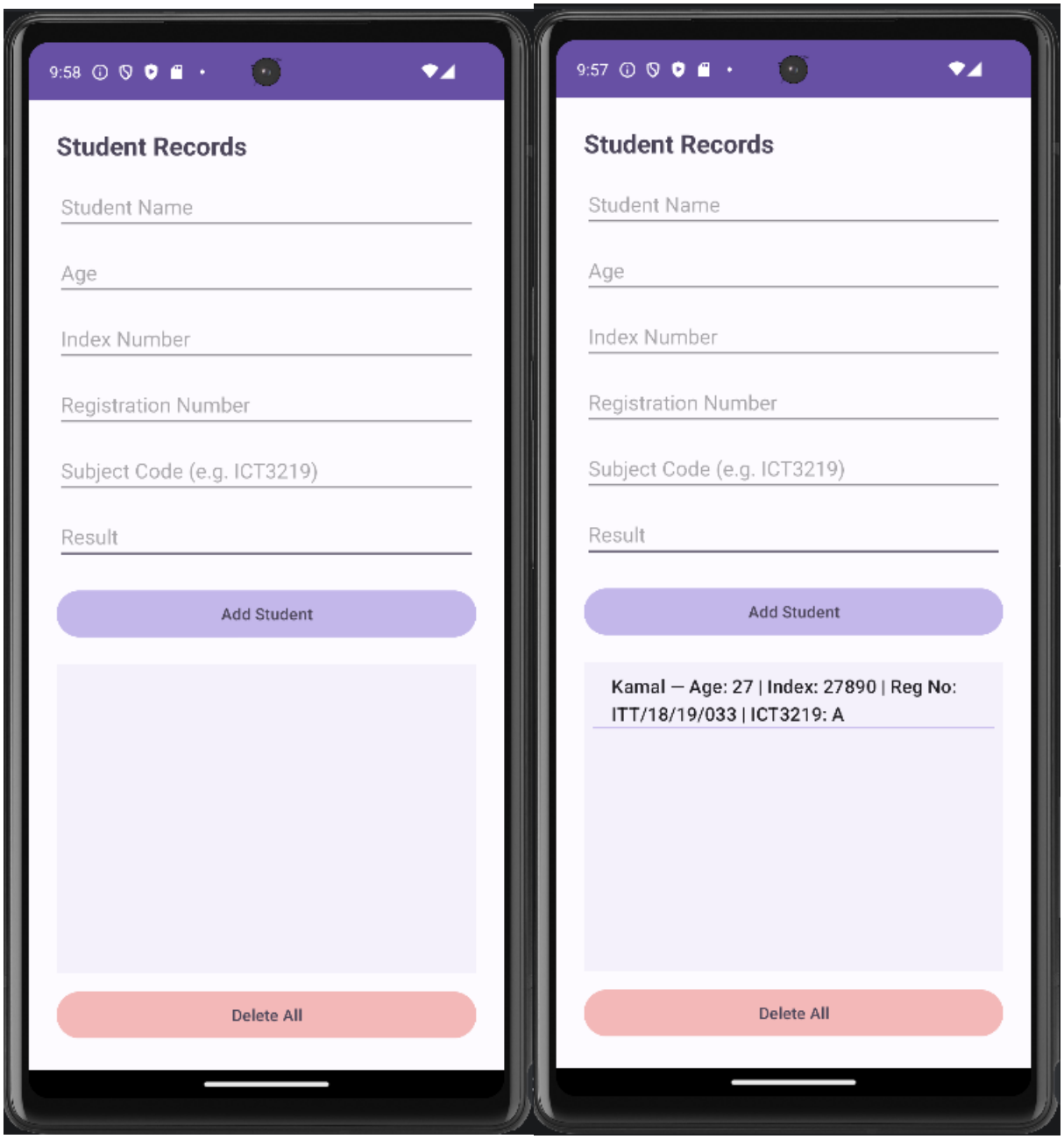
        try {
            int age = Integer.parseInt(ageStr);
            boolean success = dbHelper.addStudent(name, age, indexNumber, regNumber,
subjectCode, result);
            if (success) {
                etName.setText("");
                etAge.setText("");
                etIndexNumber.setText("");
                etRegNumber.setText("");
                etSubjectCode.setText("");
                etResult.setText("");
                loadStudents();
            } else {
                Toast.makeText(this, "Failed to add student",
Toast.LENGTH_SHORT).show();
            }
        } catch (NumberFormatException e) {
            Toast.makeText(this, "Age must be a number", Toast.LENGTH_SHORT).show();
        }
    }

    private void loadStudents() {
        studentList.clear();
        studentIds.clear();

        Cursor cursor = dbHelper.getAllStudents();
        if (cursor.moveToFirst()) {
            do {
                long id =
cursor.getLong(cursor.getColumnIndexOrThrow(DatabaseHelper.COLUMN_ID));
                String name =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.COLUMN_NAME));
                int age =
cursor.getInt(cursor.getColumnIndexOrThrow(DatabaseHelper.COLUMN_AGE));
                String indexNumber =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.COLUMN_INDEX_NUMBER));
                String regNumber =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.COLUMN_REG_NUMBER));
                String subjectCode =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.COLUMN_SUBJECT_CODE));
                String result =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.COLUMN_RESULT));

                studentIds.add(id);
                studentList.add(name + " — Age: " + age
                        + " | Index: " + indexNumber
```

```
                        + " | Reg No: " + regNumber
                        + " | " + subjectCode + ": " + result);
            } while (cursor.moveToNext());
        }
        cursor.close();
        adapter.notifyDataSetChanged();
    }
}
```



Task 6: Run and Test the App

Step 1: Run the App

- Select an emulator or connected device from the dropdown
- Click Run (green triangle)

Step 2: Test the Following Cases

- Add a student with a valid name and age → verify it appears at the top of the list

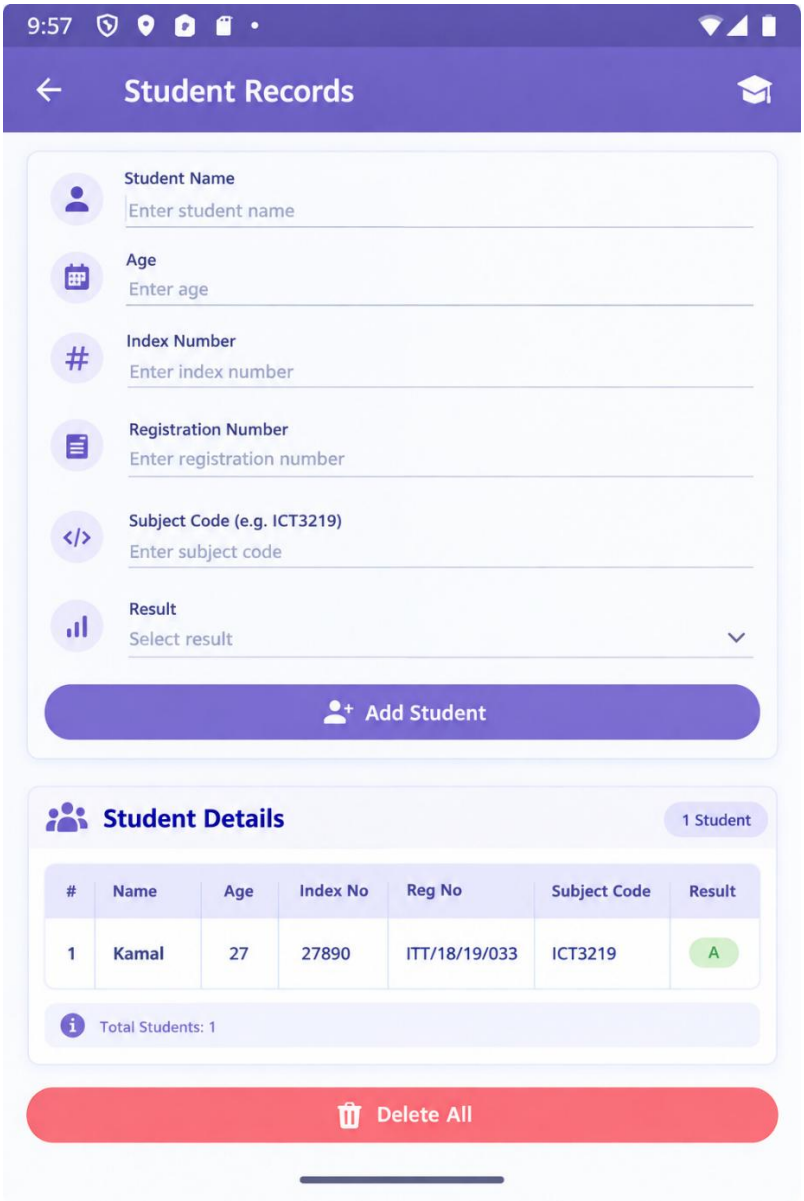
- Leave a field empty and tap Add → verify the warning Toast appears
- Enter text instead of a number for age → verify the "Age must be a number" Toast appears
- Close and reopen the app → verify previously added students are still there (this is the key difference from Practical 04 — the data persists)
- Long-press a single row → verify only that record is deleted
- Tap Delete All → verify the list becomes empty

Activity 1: Improve the UI of the Student Record App

Currently, the Student Record App displays student details as plain stacked text inside a scrollable list. Your task is to redesign the user interface to make it more organized, readable, and visually appealing.

You should:

- Display the "Add Student" form using clearly labeled fields (with icons, if possible) instead of plain unlabeled text boxes.
- Display the saved student records in a **table format** (with columns such as Name, Age, Index No, Reg No, Subject Code, and Result) instead of one long line of text per student.
- Highlight the **Result** using a colored badge (e.g. green for a pass grade).
- Show a **student count** (e.g. "3 Students") so users know how many records exist at a glance.
- Improve the color scheme, spacing, and button styling so the app looks modern and easy to use.
- Keep the existing functionality — Add Student and Delete All should still work exactly as before.



You may:

- Use the sample design provided as inspiration, or design your own layout as long as it meets the requirements above.
- Choose your own color palette, icons, and fonts.